Fase 3 — División del trabajo entre 3 desarrolladores (trabajo secuencial)

> **Nota posterior (auditoría del PR #7):** este documento describe cómo se hizo la Fase 3 y no se reescribe. Tras la auditoría se cambió lo siguiente, y manda `docs/fase-3-carga-tiempo-real.md`: la estampa de la lectura es **siempre la hora de recepción del servidor** (D6 ya no es «opcional»; `DetailRequest` no tiene `timestamp`), las lecturas de una misma orden se **serializan** con bloqueo pesimista (H-15 resuelto) y los valores **infinitos** se descartan.

Guía de trabajo para construir el **Punto 3 (activación de la carga y lecturas del caudalímetro)** entre tres personas, **una después de la otra**, en la misma rama. Dice dónde está parado cada uno respecto de la Fase 2, qué tiene que hacer, cómo hacerlo, cómo probarlo y trae el prompt para usar con Claude Code.

- **Fecha:** 27/10/2026 · **Rama:** `fase-3-carga-tiempo-real` (sale de `main` actualizado, que ya incluye las Fases 1 y 2)
- **Fase autorizada por el equipo** el 27/10/2026. Las **decisiones D1 a D10** (sección 3) quedaron **confirmadas** ese día, y también los **commits locales atómicos**.
- **Primera entrega del TP:** 10/11/2026.
- **Orden de trabajo:** primero Gonzalez, después Contreras, después Calvo. Cada uno empieza cuando el anterior **publicó** su parte.

Parte	Responsable	Usuario de GitHub	Qué construye
1	Martín González	`mgonzalez309-dev`	Modelo de datos y **activación** de la carga
2	Joaquín Contreras	`joacoContreras`	**Lecturas** del caudalímetro y **descarte**
3	Tomás Calvo	`tomas-calvo01`	Pruebas, Postman, **auditoría final**, documentación y PR
—	Benjamín Alasino	`benjaalasino`	No programa en esta fase: **revisa y aprueba el PR**

Cómo leer este documento. Todos leen las secciones 1 a 9 (contexto, contrato y forma de trabajo). Después, cada uno lee **su parte** y copia **su prompt**.

---

1. Dónde estamos parados (lo que dejó la Fase 2)

La Fase 2 construyó el Punto 2: la balanza manda la tara y la orden pasa al **estado 2** con una **contraseña de activación** de 5 dígitos. Todo esto ya existe y se reutiliza:

Ya existe	Dónde	Cómo se usa en la Fase 3
La orden en estado 2 con `tare`, `initialWeighingDateTime` y `activationPassword` (oculta con `@JsonIgnore`)	`model/Order.java`	La activación compara la contraseña recibida con `activationPassword`
El enum de estados	`model/OrderStatus.java`	Se usa `INITIAL_WEIGHING_REGISTERED` (2). Los estados 3 y 4 todavía no tienen operación
Buscar una orden por número	`OrderBusiness.load(orderNumber)` (lanza `NotFoundException`)	Primer paso de las reglas
Excepciones de negocio *checked*	`business/exception/` (`NotFound`, `Found`, `BadRequest`, `Business`, `Conflict`)	Se suma `ForbiddenException` (403)

Ya existe	Dónde	Cómo se usa en la Fase 3
El patrón de transición con comprobaciones fuera del `try`	`OrderBusiness.registerTare`	Es el molde de `activate` y de la recepción de lecturas
Un controller de integración y sus DTO	`integration/tms/controllers/OrderTmsRestController.java`, `integration/tms/model/`	Se copia el estilo para `integration/loader/`
El usuario del token	`BaseRestController.getUserLogged()`	No hace falta en esta fase (la activación no escribe historial)
Las URLs	`controllers/Constants.java`	Se agrega la del sistema de carga
Swagger	`config/OpenApiConfig.java` y anotaciones `@Operation` / `@ApiResponse`	Los endpoints nuevos se documentan igual
Postman	`docs/postman/IW3-Gas-Terminal.postman_collection.json`: 21 pedidos y 46 aserciones (carpetas 00, 01 y 02). Variables: `orderNumber`, `orderNumberTms`, `activationPassword`	Se agrega la carpeta 03 con **variables propias**
Tests	`ActivationPasswordGeneratorTest` (2 tests) y `Iw3ApplicationTests.contextLoads`	Se suman los tests del descarte

Lo que NO existe todavía y se construye ahora: los campos de la cabecera para la carga, la entidad `OrderDetail` y su tabla, la excepción de 403, la activación, la recepción de lecturas con su descarte, los endpoints del sistema de carga y sus pruebas.

2. Qué hay que construir (la Fase 3 en una página)

El **sistema de carga** (la isla donde se carga el camión) hace dos cosas:

1. **Se activa**: manda el número de orden y la contraseña que recibió la balanza. Si todo está bien, el backend le devuelve el **preset** (los kg a cargar). La orden **sigue en estado 2**.
2. **Manda lecturas** del caudalímetro, una por segundo aproximadamente: masa acumulada, densidad, temperatura, caudal y estampa de tiempo. El backend **descarta** las inválidas y **guarda** las válidas en el histórico (`order_details`), y deja en la **cabecera de la orden** los últimos valores válidos y las fechas de inicio y fin de carga.

```
Sistema de carga ──POST /api/v1/integration/loader/activation { orderNumber, password }──►
   OrderLoaderRestController ──► IOrderBusiness.activate (Parte 1) ──► Order.activationDateTime
◄── 200 { "preset": 15000.0 }                                          (la orden sigue en estado 2)

Sistema de carga ──POST /api/v1/integration/loader/orders/{n}/details { masa, densidad, temperatura, caudal,
timestamp }──►
   OrderLoaderRestController ──► IOrderDetailBusiness.add (Parte 2) ──► OrderDetail  +  cabecera de Order
◄── 200 { "saved": true }   o   200 { "saved": false, "reason": "..." }
```

2.1 Requisito de la consigna o decisión nuestra

La regla del equipo es distinguir una cosa de la otra (`CLAUDE.md`, sección 3, regla 5). **La consigna original en PDF no está disponible en las máquinas del equipo:** lo que figura como "consigna" sale de su resumen en `CLAUDE.md` (sección 4) y del plan. Calvo lo contrasta con el PDF original en la auditoría final.

Qué	Fuente
El sistema de carga manda contraseña y número de orden; se valida que la orden exista, esté en el estado esperado y la contraseña sea correcta; se devuelve el preset	Consigna (`CLAUDE.md` 4.1)
La orden sigue en estado 2 después de activar	Consigna (4.1)
Cada lectura trae masa acumulada, densidad (entre 0 y 1), temperatura, caudal y estampa de tiempo	Consigna (4.2)

Qué	Fuente
Descartar si caudal ≤ 0, masa acumulada ≤ 0 o masa menor que la anterior	Consigna (4.2)
La cabecera guarda el último valor válido de cada atributo y el detalle guarda el histórico	Consigna (4.2)
Inicio de carga = primer registro válido; fin de carga = último registro válido	Consigna (4.2)
Alcanza con guardar todas las lecturas (sin frecuencia de almacenamiento menor)	Consigna (4.2, opcional)
Los datos de detalle llegan solo en estado 2	Consigna / plan (sección 2)
Campo `activationDateTime` para distinguir "activada" de "tara registrada"	Decisión 7 del plan
Densidad `0 < d < 1`	Decisión 8 del plan
Las lecturas solo se aceptan si la orden está **activada**	Plan (`CLAUDE.md` 8.1)
403, idempotencia, respuesta `{saved, reason}`, densidad como descarte, timestamp opcional, etc.	**Decisiones nuevas D1 a D10** (sección 3)

3. Decisiones confirmadas y contrato compartido

Confirmadas por Martín el 27/10/2026. No se cambian sin avisar a los tres. Al cerrar la fase, Calvo las registra en el plan como decisiones 18 a 27 (D1 → 18, D2 → 19, ..., D10 → 27).

#	Tema	Resolución
D1	Contraseña incorrecta	**403 Forbidden** con una `ForbiddenException` nueva. El 401 se confundiría con un problema del token
D2	Activación repetida	**Idempotente**: activar de nuevo con la contraseña correcta da 200 y el mismo preset, y **no cambia** `activationDateTime`
D3	Lectura descartada	**200** con `{ "saved": false, "reason": "..." }`. Una lectura guardada responde **200** con `{ "saved": true, "reason": null }`
D4	Densidad fuera de (0, 1)	Se **descarta** (no es un 400). Los **campos ausentes** sí dan **400**
D5	Masa igual a la anterior	Es **válida**. "La anterior" es la **última masa válida** (la de la cabecera), no la última recibida
D6	`timestamp` de la lectura	**Opcional**. Si falta se usa la hora del servidor
D7	Orden de las comprobaciones	Activación: **400 (cuerpo) → 404 → 409 → 403**. Lecturas: **404 → 400 → 409 → descarte**
D8	Fin de carga	`loadingEndDateTime` es **también** la estampa de los últimos valores de la cabecera (no hay un campo duplicado)
D9	Historial de estados	La activación **no** agrega registros: no es un cambio de estado
D10	Capa de negocio	Un **`IOrderDetailBusiness` aparte**, como hace la cátedra (una interfaz de negocio por entidad). La activación va en `IOrderBusiness`

> **Decisión 11, NO confirmada:** **`GET /api/v1/orders/{n}/details`** (consulta paginada del histórico). No figura en el plan. **No se implementa** salvo que Martín la apruebe. Si se aprueba, la hace Calvo (ver "Opcional" en la Parte 3). Sin ella, el descarte se comprueba mirando la cabecera y la base (`SELECT COUNT(*)`).

3.1 Modelo de datos

`Order` (cabecera): campos nuevos, todos opcionales (la tabla `orders` ya tiene filas y `ddl-auto=update` solo agrega columnas), todos con `@Schema(READ_ONLY)` :

Campo	Tipo	Significado
`activationDateTime`	`LocalDateTime`	Cuándo se activó la carga. Nulo hasta entonces
`loadingStartDateTime`	`LocalDateTime`	Estampa de la **primera** lectura válida. No cambia después

Campo	Tipo	Significado
`loadingEndDateTime`	`LocalDateTime`	Estampa de la **última** lectura válida (D8)
`lastAccumulatedMass`	`Double`	Última masa acumulada válida (kg). Es el "producto cargado" de la Fase 5
`lastDensity`	`Double`	Última densidad válida
`lastTemperature`	`Double`	Última temperatura válida (°C)
`lastFlowRate`	`Double`	Último caudal válido (kg/h)

`OrderDetail` (histórico): tabla `order_details`

Campo	Tipo	Notas
`id`	`Long`	`IDENTITY`
`order`	`Order`	`@ManyToOne(optional = false, fetch = LAZY)`, columna `id_order`, `@JsonIgnore`
`accumulatedMass`, `density`, `temperature`, `flowRate`	`double`	Obligatorios (`nullable = false`)
`timestamp`	`LocalDateTime`	Columna `reading_timestamp` (para no usar la palabra `timestamp` del SQL)

`Order` NO tiene una lista de detalles (`@OneToMany`): con una lectura por segundo, `GET /orders/{n}` devolvería miles de filas.

3.2 Endpoints

Método y ruta	Cuerpo	Respuestas
`POST /api/v1/integration/loader/activation`	`{ "orderNumber": 90001, "password": 48213 }`	**200** `{ "preset": 15000.0 }` · 400 · 401 · **403** · 404 · 409 · 500
`POST /api/v1/integration/loader/orders/{orderNumber}/details`	`{ "accumulatedMass": 1250.5, "density": 0.52, "temperature": 18.4, "flowRate": 3600.0, "timestamp": "2026-10-28T10:15:30" }` (`timestamp` opcional)	**200** `{ "saved": true, "reason": null }` o `{ "saved": false, "reason": "..." }` · 400 · 401 · 404 · 409 · 500

`timestamp`: formato `yyyy-MM-ddTHH:mm:ss[.SSS]`, hora de Argentina, sin zona.

3.3 Clases y firmas (nombres exactos)

Pieza	Paquete (bajo `ar.edu.iua.iw3`)	Detalle
`ForbiddenException`	`business.exception`	Mismo molde que `ConflictException` (con `@Builder`)
`Constants.URL_INTEGRATION_LOADER`	`controllers`	`URL_INTEGRATION + "/loader"`
`IOrderBusiness.activate`	`business`	`Order activate(Integer orderNumber, Integer password) throws NotFoundException, BadRequestException, ConflictException, ForbiddenException, BusinessException`
`OrderDetailValidator`	`business`	Clase final de paquete, función pura: recibe masa, densidad, caudal y la masa anterior (`Double`, puede ser nula); devuelve la **lista de motivos de descarte** (vacía = válida)
`DetailResult`	`business`	Clase pública con `saved` (boolean) y `reason` (String). Constructores estáticos `saved()` y `discarded(reason)`

Pieza	Paquete (bajo `ar.edu.iua.iw3`)	Detalle
`IOrderDetailBusiness.add`	`business`	`DetailResult add(Integer orderNumber, Double accumulatedMass, Double density, Double temperature, Double flowRate, LocalDateTime timestamp) throws NotFoundException, BadRequestException, ConflictException, BusinessException`
`ActivationRequest` / `ActivationResponse`	`integration.loader.model`	`Integer orderNumber`, `Integer password` / `Double preset`
`DetailRequest` / `DetailResponse`	`integration.loader.model`	`Double` ×4 y `LocalDateTime timestamp` / `boolean saved`, `String reason`
`OrderLoaderRestController`	`integration.loader.controllers`	`@RequestMapping(Constants.URL_INTEGRATION_LOADER)`, `@Tag("Integración sistema de carga")`, extiende `BaseRestController`

Los pedidos usan `Double` y `Integer` (objetos) y no `double` / `int`. Es a propósito: con primitivos, un campo ausente llegaría como `0` y se **descartaría** en vez de dar **400** (D4).

3.4 Reglas, en orden

Activación (`IOrderBusiness.activate`), con `@Transactional(rollbackFor = Exception.class)` y las comprobaciones **fuera del `try`**:

1. `orderNumber` o `password` ausentes → **400**.
2. `load(orderNumber)` → si no existe, **404**.
3. Estado distinto de 2 → **409**.
4. Contraseña distinta de `activationPassword` → **403**.
5. Si `activationDateTime` es nulo, se guarda la hora actual (**idempotente**, D2). Devuelve la orden; el controller responde el `preset`.

Lecturas (`IOrderDetailBusiness.add`), `@Transactional(rollbackFor = Exception.class)`:

1. `load(orderNumber)` → **404**.
2. Algún campo numérico ausente → **400**.
3. Estado distinto de 2 **o** orden sin activar (`activationDateTime` nulo) → **409**.
4. Motivos de descarte (`OrderDetailValidator`): si hay alguno → se devuelve `DetailResult.discarded(...)` **sin escribir nada**.
5. Si es válida (dentro del `try`): se guarda el `OrderDetail` y se actualiza la cabecera con los cuatro últimos valores, `loadingStartDateTime` (solo si es nulo) y `loadingEndDateTime` (siempre). Estampa = la del cliente o, si falta, la hora del servidor.

Descarte (todas las condiciones se evalúan y los motivos se juntan con `"; "`):

Condición	Motivo (texto)
`!(caudal > 0)`	`el caudal (flowRate) debe ser mayor que 0`
`!(masa > 0)`	`la masa acumulada (accumulatedMass) debe ser mayor que 0`
masa anterior no nula y `masa < anterior`	`la masa acumulada es menor que la anterior (<valor>)`
`!(densidad > 0 && densidad < 1)`	`la densidad (density) debe estar entre 0 y 1, sin incluirlos`

Se usa la forma `!(x > 0)` y no `x <= 0` para que un `NaN` también se descarte. **La temperatura no tiene regla de descarte** (puede ser negativa).

3.5 Mensajes del contrato

Calvo arma las aserciones de Postman sobre **fragmentos** de estos textos (con `include`), no sobre el texto completo.

Caso	Código	Mensaje
Activación sin `orderNumber` o `password`	400	`orderNumber y password son obligatorios`
Activación de orden fuera del estado 2	409	`La orden <n> no está lista para activar (estado <e>)`
Contraseña incorrecta	403	`La contraseña de activación es incorrecta` (**no repite ninguna contraseña**)
Lectura con campos ausentes	400	`accumulatedMass, density, temperature y flowRate son obligatorios`
Lectura de orden no activada o fuera de estado 2	409	`La orden <n> no acepta lecturas: debe estar en estado 2 y activada (estado <e>, activada=<true/false>)`
Orden inexistente	404	`No se encuentra la Orden numero=<n>`

4. Cuidados y trampas (leelos antes de empezar)

1. **Campos ausentes frente a campos inválidos.** Ausente = 400, inválido = se descarta. Por eso `Double` / `Integer` en los pedidos y la comprobación de nulos **antes** del descarte.
2. **Comprobaciones fuera del `try`.** Si el `catch (Exception)` las envolviera, un 404, 400, 409 o 403 saldría como 500 (lección de las Fases 1 y 2).
3. **`@Transactional(rollbackFor = Exception.class)`.** Las excepciones de negocio son *checked* y Spring no revierte por ellas.
4. **Descartar no es un error.** No se lanza ninguna excepción: se devuelve el resultado y no se escribe nada.
5. **La contraseña no se muestra ni se registra.** Ni en los mensajes de error, ni en los logs, ni en el JSON de la orden (`@JsonIgnore` de la Fase 2 sigue vigente).
6. **No agregar una lista de detalles a `Order`.** Infla `GET /orders/{n}` y complica la serialización.
7. **`OrderDetail.order` va con `@JsonIgnore` y `LAZY`.** Si no, el JSON arrastra la orden completa.
8. **Las columnas nuevas de `orders` deben ser opcionales.** Si no, MySQL falla al agregarlas sobre filas existentes.
9. **Las órdenes que ya están en estado 2 sin `activationDateTime`** (las de pruebas anteriores) son órdenes **no activadas**: se pueden activar con su contraseña.
10. **Estampa de tiempo.** Es `LocalDateTime` sin zona. En una prueba descartable previa (que no forma parte del repo) se observó que un valor con `Z` al final **se acepta y se descarta la zona sin avisar**. Es una limitación conocida: las pruebas usan valores sin zona.
11. **Eficiencia (llegan datos cada ~1 s).** La versión simple es suficiente: cargar la orden, guardar el detalle y guardar la cabecera. La masa anterior sale de la **cabecera**; no se recorre el histórico. La Parte 2 mide 100 lecturas seguidas y registra el resultado (orientativo: unos 100 ms o menos cada una en local). Solo si se mide un problema real se evalúan las alternativas de la cátedra (clase 7: `@Modifying @Query` y proyecciones).
12. **A verificar en la Parte 2 (hipótesis, no medido).** La actualización de la cabecera se envía a la base al **confirmar** la transacción, es decir, **después** del `try/catch` del método. Un error de base en ese punto podría salir como un 500 con el formato por defecto de Spring y no con `StandartResponse`. Verificarlo y, si pasa, evaluar `saveAndFlush` dentro del `try`.
13. **Concurrencia.** Dos lecturas simultáneas de la misma orden podrían pisarse al comparar la masa anterior. Se acepta para el parcial, igual que H-12 y N-2.
14. **Seguridad conocida.** La contraseña es de 5 dígitos (90.000 combinaciones) y no hay límite de intentos; el token vence a la hora, así que una carga larga exige volver a loguearse. Se registran como limitaciones; se resuelven en la etapa de seguridad del final.
15. **La Fase 4 reutiliza la comprobación de las lecturas.** Al pasar la orden a estado 3, las lecturas deben rechazarse con 409 **sin tocar código de la Fase 3**: por eso la comprobación es "estado 2 y activada".

16. **Postman.** Las variables de la Fase 3 son propias (`orderNumberLoad`, `activationPasswordLoad`, ...). Usar `orderNumber` o `activationPassword` rompe las aserciones de las Fases 1 y 2.

5. Fuera de alcance de la Fase 3

- **Fase 4:** cerrar la orden (estado 3).
- **Fase 5:** pesaje final, conciliación y promedios (`AVG`).
- **Examen final:** roles, alarma de temperatura por mail (eventos de la clase 9) y frontend.
- **Opcional del parcial:** frecuencia de almacenamiento distinta de la de recepción (se guardan todas las lecturas válidas).
- `GET /orders/{n}/details` (decisión 11, sin confirmar).
- Límite de intentos de la contraseña y renovación de token.

Los campos de la cabecera quedan listos para el final (carga actual, últimos valores, tiempo transcurrido = ahora − inicio de carga), pero **no se implementa nada del frontend ni de las alarmas**.

6. Criterios de aceptación de la fase

#	Criterio
1	Activación correcta: 200 con `{ "preset": ... }`, `activationDateTime` guardado y el estado sigue en 2
2	Activación repetida con la contraseña correcta: 200, mismo preset y `activationDateTime` sin cambios
3	Contraseña incorrecta: 403; la orden no se activa y el mensaje no repite ninguna contraseña
4	Orden inexistente: 404 · orden fuera del estado 2: 409 · `orderNumber` o `password` ausentes: 400 · sin token: 401
5	La activación no agrega registros al historial de estados
6	Lectura válida: 200 `{ "saved": true }`; se guarda en `order_details` y la cabecera guarda los últimos valores
7	`loadingStartDateTime` es la estampa de la primera lectura válida (y no cambia) y `loadingEndDateTime` la de la última
8	Lectura inválida (caudal ≤ 0, masa ≤ 0, masa menor que la anterior o densidad fuera de (0, 1)): 200 `{ "saved": false, "reason": ... }`; no se guarda ni toca la cabecera
9	Una masa igual a la anterior es válida
10	Campo ausente, JSON malformado o valor no numérico: 400
11	Lectura de orden no activada o fuera del estado 2: 409 · inexistente: 404 · sin token: 401
12	Atomicidad: el detalle y la cabecera se guardan juntos o no se guarda ninguno
13	`GET /orders/{n}` muestra los campos de carga, sigue ocultando la contraseña y no trae lista de detalles
14	Las tablas y columnas nuevas se crean sobre una base con órdenes existentes, sin error
15	Swagger documenta los dos endpoints con sus respuestas
16	Postman: la carpeta 03 pasa con newman y las carpetas 01 y 02 siguen pasando (46 aserciones)
17	Tests unitarios (descarte y generador) y `./mvnw -B test` en verde
18	Eficiencia: 100 lecturas consecutivas sin degradación apreciable (criterio orientativo, la medición queda registrada)
19	Informe y guía de la Fase 3, documentación corregida (apartado E de la Parte 3), PDF y **auditoría final** registrada

7. Reparto y propiedad de archivos

Como el trabajo es **secuencial**, no hay riesgo de que dos personas editen a la vez. Aun así, cada archivo tiene un dueño y los que tocan dos partes se **extienden sin reescribir** lo anterior.

Archivo	Acción	Dueño
`model/Order.java`	Modificar (7 campos)	**Parte 1**

Archivo	Acción	Dueño
`model/OrderDetail.java`, `model/persistence/OrderDetailRepository.java`	Crear	**Parte 1**
`business/exception/ForbiddenException.java`	Crear	**Parte 1**
`controllers/Constants.java`	Modificar (1 línea)	**Parte 1**
`business/IOrderBusiness.java`, `business/OrderBusiness.java`	Agregar `activate`	**Parte 1**
`integration/loader/model/ActivationRequest.java`, `ActivationResponse.java`	Crear	**Parte 1**
`integration/loader/controllers/OrderLoaderRestController.java`	Crear con la activación	**Parte 1**
`business/OrderDetailValidator.java` y su test	Crear	**Parte 2**
`business/DetailResult.java`, `IOrderDetailBusiness.java`, `OrderDetailBusiness.java`	Crear	**Parte 2**
`integration/loader/model/DetailRequest.java`, `DetailResponse.java`	Crear	**Parte 2**
`OrderLoaderRestController.java`	**Agregar** el método de lecturas y la inyección (sin tocar la activación)	**Parte 2**
`docs/postman/IW3-Gas-Terminal.postman_collection.json`	Agregar la carpeta 03	**Parte 3**
`docs/fase-3-carga-tiempo-real.md` y su PDF	Crear	**Parte 3**
`CLAUDE.md`, `docs/plan-de-trabajo.md`, `docs/guia-estudio-src.md`, `docs/fase-2-pesaje-inicial.md`	Corregir y actualizar al cerrar	**Parte 3**
Este documento (bloque "Estado de la Parte N") y filas de `docs/guia-estudio-src.md` para sus archivos	Agregar	**Cada parte**, lo suyo

No se commitean las carpetas `.postman/` ni `postman/` (son el vínculo con el espacio de trabajo de Postman de cada uno).

Estimación (a ajustar): Parte 1 ≈ 2 a 2,5 h · Parte 2 ≈ 3 a 3,5 h · Parte 3 ≈ 5 a 6 h (es la más larga por la escritura y la auditoría; se alivia porque las Partes 1 y 2 dejan sus bloques y sus filas de la guía listos).

8. Orden de trabajo y traspasos

```
Etapa 0   Gonzalez crea la rama y publica este documento            ← los otros dos esperan este push
Parte 1   Gonzalez implementa, verifica, pushea y AVISA a Contreras
Parte 2   Contreras trae la rama, implementa, verifica, pushea y AVISA a Calvo
Parte 3   Calvo trae la rama, hace Postman + auditoría + documentación, abre el PR
Cierre    Revisión del PR: Gonzalez, Contreras y Benjamín aprueban → merge
```

Propuesta de fechas (a ajustar entre ustedes): Parte 1 el miércoles 28/10 · Parte 2 el jueves 29/10 · Parte 3 y PR el viernes 30/10 · merge hasta el lunes 2/11. Eso deja del 2 al 9/11 para las Fases 4 y 5 antes de la entrega del 10/11.

Cada parte, al terminar, hace tres cosas antes de pasar la posta:

1. Agrega al final de este documento el bloque **"Estado de la Parte N"** (ver el modelo en el Anexo B): qué quedó hecho, commits, qué se verificó con resultados reales, qué debe tener en cuenta el siguiente.
2. Agrega a `docs/guia-estudio-src.md` las filas de sus archivos nuevos, marcados como `F3`.
3. Pushea y le avisa al siguiente.

Mientras esperás tu turno podés preparar sin pushear: Contreras puede escribir y probar localmente el validador de descarte (no depende de nada); Calvo puede diseñar los casos de Postman y la matriz de auditoría.

Revisión cruzada (para la evaluación individual): Gonzalez revisa a Contreras, Contreras revisa a Calvo y Calvo revisa a Gonzalez (esto último dentro de la auditoría). **Benjamín**, que no programa en esta fase, revisa el PR completo y lo aprueba: así se llega a las 3 aprobaciones del acuerdo (el autor, Calvo, no cuenta).

9. Cómo trabajar con Git y cómo probar

```
# Solo Gonzalez, una vez:
git checkout main && git pull
git checkout -b fase-3-carga-tiempo-real
# ... commits de la documentación ...
git push -u origin fase-3-carga-tiempo-real

# Contreras y Calvo, cuando el anterior avise que publicó:
git fetch
git checkout fase-3-carga-tiempo-real
git pull --rebase

# Todos, antes de CADA push:
git pull --rebase
git push
```

Regla	Detalle
Commits	**Atómicos**, de **100 líneas o menos**, en español, formato `tipo(ámbito): descripción` (`feat`, `fix`, `docs`, `test`, `build`, `refactor`). Están **autorizados los commits locales**
Excepción de las 100 líneas	Documentos nuevos completos y el JSON de Postman: un commit aparte por archivo
Coautoría	**Sin** `Co-Authored-By: Claude` ni "Generated with Claude Code"
Push	Solo cuando el responsable lo pide, y siempre después de `git pull --rebase`
`main`	Nadie pushea a `main`. Entra por Pull Request
Force push	Prohibido
Alcance	No implementar la Fase 4 ni la 5, no refactorizar fuera de la fase y no agregar dependencias

Si el push es rechazado ("non-fast-forward"): alguien pusheó antes. `git pull --rebase` y volver a pushear. **Si hay un conflicto**, alguien editó un archivo ajeno: avisen y decidan entre los dos dueños; no lo resuelvan a ciegas.

Probar en tu máquina:

```
docker compose up -d                 # la base (contenedor iw3-gas-mysql)
./mvnw spring-boot:run               # la app en http://localhost:8080
# Solo la primera vez, con la app ya iniciada (PowerShell):
Get-Content docs/sql/seed-dev-users.sql | docker exec -i iw3-gas-mysql mysql -uroot -proot iw3_db
```

Usuario de desarrollo: `admin` / `admin123`. Swagger: `http://localhost:8080/swagger-ui.html`.

```
TOKEN=$(curl -s -X POST localhost:8080/api/v1/login -d "username=admin&password=admin123")

# Activar (la contraseña sale de la tara: carpeta 02 de Postman, o POST .../tms/orders/{n}/tare)
curl -i -X POST localhost:8080/api/v1/integration/loader/activation \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"orderNumber": 90001, "password": 48213}'

# Mandar una lectura
curl -i -X POST localhost:8080/api/v1/integration/loader/orders/90001/details \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"accumulatedMass": 100.0, "density": 0.52, "temperature": 18.4, "flowRate": 3600.0, "timestamp": "2026-10-
28T10:00:00"}'

# Mirar la base
docker exec iw3-gas-mysql mysql -uroot -proot iw3_db -e "SELECT order_number, status, activation_date_time,
loading_start_date_time, loading_end_date_time, last_accumulated_mass FROM orders ORDER BY id DESC LIMIT 5;"
docker exec iw3-gas-mysql mysql -uroot -proot iw3_db -e "SELECT id, id_order, accumulated_mass, density,
temperature, flow_rate, reading_timestamp FROM order_details ORDER BY id DESC LIMIT 10;"
```

En PowerShell usá `curl.exe` y comillas simples para el JSON, o directamente Swagger.

Parte 1 — Modelo y activación

Responsable: Martín González · **Estimación:** ≈ 2 a 2,5 h · **Es la primera: las otras dos dependen de su push.**

Punto de partida (qué heredás de la Fase 2)

`Order` tiene la tara y la contraseña, pero **no** los campos de carga ni la fecha de activación. **No existe** ninguna entidad para las lecturas ni excepción de 403. `Constants` tiene la URL de SAP y la de TMS, **no la del sistema de carga**. No hay paquete `integration/loader`.

Objetivo

Dejar el modelo de datos completo de la fase (cabecera e histórico) y resolver la **activación**: el sistema de carga se activa con la contraseña y recibe el preset. Las otras partes se apoyan en lo que dejes.

Archivos

Todos bajo `src/main/java/ar/edu/iua/iw3/`.

Archivo	Acción
`model/Order.java`	Agregar los 7 campos de la sección 3.1
`model/OrderDetail.java`	Crear (sección 3.1)
`model/persistence/OrderDetailRepository.java`	Crear, extendiendo `JpaRepository<OrderDetail, Long>` (por ahora sin métodos propios)
`business/exception/ForbiddenException.java`	Crear (molde de `ConflictException`)
`controllers/Constants.java`	Agregar `URL_INTEGRATION_LOADER`
`business/IOrderBusiness.java` y `OrderBusiness.java`	Agregar `activate` (sección 3.4)
`integration/loader/model/ActivationRequest.java` y `ActivationResponse.java`	Crear
`integration/loader/controllers/OrderLoaderRestController.java`	Crear con **solo** el endpoint de activación

Qué hacer, paso a paso

Paso 0 — Abrir la rama y publicar la documentación (primero de todo): ver el prompt. Después avisá a Contreras y a Calvo que la rama existe (ellos aún no empiezan).

Paso 1 — `Order`. Siete campos opcionales con `@Schema(accessMode = READ_ONLY)` y un comentario breve cada uno. Tipos de la sección 3.1.

Paso 2 — `OrderDetail` y su repositorio. `@ManyToOne` a `Order` con `LAZY`, `@JsonIgnore` y columna `id_order`; cuatro `double` obligatorios; `timestamp` con `@Column(name = "reading_timestamp")`. Constructor completo (`@AllArgsConstructor`) para que la Parte 2 lo use.

Paso 3 — `ForbiddenException` y la URL.

Paso 4 — `activate`. Las cinco reglas de la sección 3.4, en ese orden, con `@Transactional(rollbackFor = Exception.class)` y las comprobaciones **fuera del** `try`. El mensaje de 403 **no repite ninguna contraseña**. Recordá importar `ForbiddenException` en `IOrderBusiness` (y en el controller).

Paso 5 — DTO y endpoint. Pedido con `Integer`; respuesta con `Double preset`. En el controller: `@Tag`, `@Operation` y `@ApiResponse` (200, 400, 403, 404, 409), y la traducción de errores con `IStandartResponseBusiness`: 400, 404, 409, **403** y 500. Mismo estilo que `OrderTmsRestController`.

Paso 6 — Cierre de la parte: bloque "Estado de la Parte 1" y filas de la guía de `src`.

Plan de commits sugerido (todos de menos de 100 líneas):

1. `feat(model): agrega a la orden los campos de activación y de carga`
2. `feat(model): agrega la entidad OrderDetail y su repositorio`
3. `feat(business): agrega ForbiddenException para la contraseña incorrecta`
4. `feat(controllers): agrega la URL de integración del sistema de carga`
5. `feat(business): agrega la activación de la orden`
6. `feat(integration): agrega el pedido y la respuesta de la activación`
7. `feat(integration): agrega el endpoint de activación del sistema de carga`
8. `docs(fase-3): registra el estado de la Parte 1 y actualiza el mapa de código`

Cómo probar

Primero necesitás una orden en estado 2 con su contraseña: corré en Postman la carpeta 00 y los dos primeros pedidos de la 02 (`Crear orden...` y `Tara válida`; la contraseña queda en la variable `activationPassword`), o usá Swagger o `curl`.

Caso	Esperado
Cuerpo `{}`, sin `password` o sin `orderNumber`	400
Orden inexistente	404
Orden en estado 1	409
Contraseña incorrecta	403, la orden **no** queda activada y el mensaje no incluye la contraseña
Contraseña correcta	200 `{ "preset": 15000.0 }`; `GET /orders/{n}` muestra `activationDateTime` y `status` sigue en 2
Activar otra vez con la contraseña correcta	200, mismo preset, `activationDateTime` **igual**
`password` no numérica (`"abc"`) o JSON malformado	400
Sin token	401

También comprobá:

- `GET /orders/{n}` muestra los campos nuevos (en `null` hasta que haya lecturas), **no** muestra `activationPassword` y no trae ninguna lista de detalles.
- **El historial** (`/status-log`) sigue con 2 registros después de activar (D9).
- **La app arranca sobre tu base con órdenes**, y Hibernate agrega las columnas nuevas (opcionales) y crea `order_details`.
- **Regresión:** `npx newman@6 run docs/postman/IW3-Gas-Terminal.postman_collection.json` sigue dando las 46 aserciones sin fallas.
- **Swagger** muestra `POST /integration/loader/activation` con sus respuestas.
- `./mvnw -B -q -DskipTests compile` sin errores.

Criterio de terminado

- Los casos de la tabla se cumplen y las comprobaciones de arriba también.

- Commits locales hechos y **publicados a la rama** (cuando lo pidas).
- Bloque "Estado de la Parte 1" escrito, y **Contreras avisado**.

No tocar

Nada de lecturas ni de descarte (Parte 2), Postman, `CLAUDE.md`, el plan ni los informes (Parte 3). Si necesitás un cambio ajeno, avisá.

Qué le dejás a Contreras

La entidad `OrderDetail` con su constructor completo, `OrderDetailRepository`, los siete campos de `Order`, el controller `OrderLoaderRestController` con la activación funcionando, y `ForbiddenException`.

Prompt para Claude Code (Parte 1)

```
Contexto: repo iw3-gas-terminal-backend (Spring Boot 4, Java 21, MySQL). Estamos en la Fase 3 (punto 3: activación de la carga y lecturas del caudalímetro), rama fase-3-carga-tiempo-real. El trabajo es SECUENCIAL: primero Gonzalez (esta parte), después Contreras y después Calvo. El equipo autorizó la fase el 27/10/2026 y confirmó las decisiones D1 a D10 del documento de división, además de los commits locales atómicos. Las líneas de CLAUDE.md que dicen que la Fase 2 espera su PR o que la Fase 3 está pendiente quedaron desactualizadas: no las edites (las corrige Calvo al cerrar la fase).

Leé primero: CLAUDE.md, docs/plan-de-trabajo.md y docs/fase-3-division-de-trabajo.md (secciones 1 a 6 y "Parte 1").

Soy Martín González y hago la PARTE 1 (modelo y activación).

Paso 0 (primero): desde main actualizado (git checkout main && git pull) creá la rama fase-3-carga-tiempo-real. Commiteá, en commits separados y SIN incluir las carpetas .postman/ ni postman/: (a) docs/fase-3-division-de-trabajo.md y su PDF; (b) docs/pdf/README.md. No pushees hasta que yo te lo pida.

Después implementá SOLO estos archivos (rutas bajo src/main/java/ar/edu/iua/iw3/):
1. model/Order.java: agregar activationDateTime, loadingStartDateTime, loadingEndDateTime (LocalDateTime) y lastAccumulatedMass, lastDensity, lastTemperature, lastFlowRate (Double). Todos opcionales (la tabla orders ya tiene filas), con @Schema(accessMode = READ_ONLY) y un comentario breve.
2. model/OrderDetail.java: entidad de la tabla order_details con id (IDENTITY), order (@ManyToOne(optional = false, fetch = LAZY), @JsonIgnore, columna id_order), accumulatedMass, density, temperature y flowRate (double, nullable = false) y timestamp (LocalDateTime, @Column(name = "reading_timestamp", nullable = false)). Lombok como el resto de las entidades, con @AllArgsConstructor. NO agregues ninguna lista de detalles en Order.
3. model/persistence/OrderDetailRepository.java: interfaz que extiende JpaRepository<OrderDetail, Long>, sin métodos propios.
4. business/exception/ForbiddenException.java: mismo molde que ConflictException (con @Builder); se traduce a 403.
5. controllers/Constants.java: agregar URL_INTEGRATION_LOADER = URL_INTEGRATION + "/loader".
6. business/IOrderBusiness.java y business/OrderBusiness.java: agregar Order activate(Integer orderNumber, Integer password) throws NotFoundException, BadRequestException, ConflictException, ForbiddenException, BusinessException, con @Transactional(rollbackFor = Exception.class) y las comprobaciones FUERA del try, en este orden: (a) orderNumber o password nulos -> 400 con el mensaje "orderNumber y password son obligatorios"; (b) load(orderNumber) -> 404; (c) estado distinto de INITIAL_WEIGHING_REGISTERED -> 409 con "La orden <n> no está lista para activar (estado <e>)"; (d) password distinta de activationPassword -> 403 con "La contraseña de activación es incorrecta" (el mensaje NO repite ninguna contraseña); (e) si activationDateTime es nulo, guardarlo con la hora actual (idempotente: si ya estaba, no cambia). Devuelve la orden. La activación NO escribe en el historial de estados. Cualquier otra falla -> log.error y BusinessException.
7. integration/loader/model/ActivationRequest.java (Integer orderNumber, Integer password) y ActivationResponse.java (Double preset), con @Schema y ejemplos, como TareRequest y TareResponse.
8. integration/loader/controllers/OrderLoaderRestController.java: @RestController en Constants.URL_INTEGRATION_LOADER, extiende BaseRestController, con @Tag("Integración sistema de carga"). SOLO el endpoint POST /activation (consumes y produces JSON) con @Operation y @ApiResponse (200, 400, 403, 404, 409). Responde 200 con new ActivationResponse(order.getPreset()). Traducción con IStandartResponseBusiness.build(status, e, e.getMessage()): BadRequestException -> 400, NotFoundException -> 404, ConflictException -> 409, ForbiddenException -> 403, BusinessException -> 500. Mismo estilo que OrderTmsRestController.

NO toques: nada de lecturas ni de descarte (Parte 2: OrderDetailValidator, DetailResult, IOrderDetailBusiness, OrderDetailBusiness, DetailRequest, DetailResponse), el JSON de Postman, CLAUDE.md, docs/plan-de-trabajo.md ni los informes (Parte 3). No implementes la Fase 4 ni la 5. Si creés que hace falta cambiar algo ajeno, avisame en vez de hacerlo.

Reglas: metodología de la cátedra (capas, I*Business, excepciones checked, StandartResponse, Constants); identificadores en inglés y comentarios en español. Commits atómicos de 100 líneas o menos, en español, formato tipo(ámbito): descripción, SIN Co-Authored-By de Claude ni "Generated with Claude Code" (están autorizados los commits locales). No hagas push, merge ni force push salvo que yo lo pida; cuando lo pida, primero git pull --rebase. No agregues dependencias ni refactorices fuera de la fase. Antes de editar, mostrame tu plan de archivos y commits y esperá mi OK.

Verificá de verdad y reportá solo lo que ejecutaste: compilá (./mvnw -B -q -DskipTests compile); con la base levantada arrancá la app sobre una base que ya tenga órdenes y confirmá que Hibernate crea order_details y agrega las columnas opcionales a orders; creá una orden en estado 2 con su contraseña (carpeta 02 de Postman o Swagger) y probá la activación: cuerpo vacío, sin password y sin orderNumber (400), orden inexistente (404), orden en estado 1 (409), contraseña incorrecta (403, la orden NO queda activada y el mensaje no incluye la contraseña), contraseña correcta (200 con el preset, activationDateTime guardado, estado sigue en 2), repetir la activación
```

```
(200, mismo preset, activationDateTime sin cambios), password "abc" y JSON malformado (400) y sin token (401).
Comprobá que GET /api/v1/orders/{n} muestra los campos nuevos en null, no muestra activationPassword y no trae
lista de detalles, que /status-log sigue con 2 registros, que Swagger muestra el endpoint, y corré npx newman@6
run docs/postman/IW3-Gas-Terminal.postman_collection.json: tienen que seguir pasando las 46 aserciones. Si algo
falla, decilo.

Al terminar: agregá al final de docs/fase-3-division-de-trabajo.md el bloque "Estado de la Parte 1" (qué quedó
hecho, commits, qué verificaste con resultados reales y qué debe tener en cuenta Contreras) y las filas de tus
archivos en docs/guia-estudio-src.md marcadas F3 (commits aparte). No pushees hasta que yo lo pida.
```

Parte 2 — Lecturas del caudalímetro y descarte

Responsable: Joaquín Contreras · **Estimación:** ≈ 3 a 3,5 h · **Es la parte con más lógica de la fase.**

Punto de partida (qué heredás de la Parte 1)

Antes de empezar, traé la rama y comprobá que están: la entidad `OrderDetail` y su repositorio, los siete campos de `Order`, `ForbiddenException`, `Constants.URL_INTEGRATION_LOADER` y `OrderLoaderRestController` con la activación funcionando. Si falta algo, **no empieces**: avisá a Gonzalez.

Objetivo

Recibir las lecturas del caudalímetro: **descartar** las inválidas (sin escribir nada) y **guardar** las válidas en el histórico, actualizando la cabecera de la orden (últimos valores, inicio y fin de carga).

Archivos

Todos bajo `src/main/java/ar/edu/iua/iw3/` salvo el test.

Archivo	Acción
`business/OrderDetailValidator.java`	Crear. Función pura que devuelve la lista de motivos de descarte (sección 3.4)
`src/test/java/ar/edu/iua/iw3/business/OrderDetailValidatorTest.java`	Crear. JUnit 5 **sin** `@SpringBootTest`
`business/DetailResult.java`	Crear (`saved`, `reason`, `saved()` y `discarded(reason)`)
`business/IOrderDetailBusiness.java`	Crear con **solo** `add`
`business/OrderDetailBusiness.java`	Crear
`integration/loader/model/DetailRequest.java` y `DetailResponse.java`	Crear
`integration/loader/controllers/OrderLoaderRestController.java`	**Agregar** el método de lecturas y la inyección de `IOrderDetailBusiness`

Qué hacer, paso a paso

Paso 1 — Validador y su test. Reglas de descarte de la sección 3.4, todas evaluadas y con los textos del contrato. El test cubre: lectura válida, **masa igual a la anterior** (válida), caudal 0 y negativo, masa 0 y negativa, masa menor que la anterior, densidad 0, 1, mayor que 1 y negativa, `NaN`, y varios motivos juntos.

Paso 2 — `DetailResult` y la interfaz.

Paso 3 — `OrderDetailBusiness.add`. Cuatro dependencias: `IOrderBusiness` (para `load`), `OrderRepository` (para guardar la cabecera), `OrderDetailRepository` y el validador. Las reglas de lecturas de la sección 3.4, en ese orden y

con las tres primeras comprobaciones **fuera del** `try`. La masa anterior sale de `order.getLastAccumulatedMass()`. La estampa es `timestamp` o, si es nulo, `LocalDateTime.now()`. Todo en `@Transactional(rollbackFor = Exception.class)`.

Paso 4 — DTO. `DetailRequest` con `Double` ×4 y `LocalDateTime timestamp` (opcional); `DetailResponse` con `boolean saved` y `String reason`. Con `@Schema` y ejemplos.

Paso 5 — Endpoint. `POST /orders/{orderNumber}/details` en `OrderLoaderRestController`, siempre **200** para guardada o descartada, y 404, 400, 409 y 500 para los errores. Agregalo **sin modificar** el método de activación.

Paso 6 — Cierre: bloque "Estado de la Parte 2" y filas de la guía de `src`.

Plan de commits sugerido:

1. `feat(business): agrega las reglas de descarte de las lecturas`
2. `test(business): prueba las reglas de descarte`
3. `feat(business): agrega DetailResult`
4. `feat(business): agrega la interfaz del negocio de lecturas`
5. `feat(business): implementa la recepción de lecturas y la actualización de la cabecera`
6. `feat(integration): agrega el pedido y la respuesta de las lecturas`
7. `feat(integration): agrega el endpoint de lecturas del caudalímetro`
8. `docs(fase-3): registra el estado de la Parte 2 y actualiza el mapa de código`

Cómo probar

Necesitás una orden **activada** (la activación es de la Parte 1).

Secuencia sugerida para una orden activada, con timestamps fijos:

#	Lectura	Esperado
1	masa 100, densidad 0.5, temp 20, caudal 3000	`saved: true`
2	masa 250	`saved: true`. Cabecera: `lastAccumulatedMass` 250; inicio = estampa de la 1; fin = estampa de la 2
3	masa 200 (< 250)	`saved: false`, motivo "menor que la anterior"
4	caudal 0 y caudal −5	`saved: false`
5	masa 0 y masa −1	`saved: false`
6	densidad 0, 1, 1.5 y −0.2	`saved: false`
7	masa 250 (igual)	`saved: true`
8	temperatura negativa con el resto válido	`saved: true`
9	sin `timestamp`	`saved: true`; el fin de carga queda con la hora del servidor y el inicio **no cambia**
10	falta un campo, `{}`, JSON malformado, `"abc"` en un número, `timestamp` con formato inválido	400
11	orden inexistente	404
12	orden en estado 1 o en estado 2 sin activar	409
13	sin token	401

Comprobá además:

- **Los descartes no tocaron nada:** la cabecera sigue con los últimos valores válidos y `SELECT COUNT(*) FROM order_details WHERE id_order = <id>` coincide con la cantidad de lecturas guardadas.
- `GET /orders/{n}` muestra los campos de la cabecera y no trae lista de detalles.

- **Eficiencia (D, trampa 11):** mandá **100 lecturas seguidas** (un script corto) con masa creciente y registrá el tiempo total y el promedio por lectura. Orientativo: unos 100 ms o menos cada una en local. Anotá el resultado real en tu bloque de estado.
- **Falla al confirmar (trampa 12):** comprobá qué responde la app si la base falla justo al actualizar la cabecera (por ejemplo, achicando **temporalmente** una columna `last_...` en tu base **local** para forzar un error, y restaurándola después). Si la respuesta **no** es un `StandartResponse`, evaluá `saveAndFlush` dentro del `try` y dejá la conclusión anotada.
- `./mvnw -B test` (con la base arriba) en verde, y las carpetas 01 y 02 de Postman con newman siguen pasando.
- Swagger muestra el endpoint con sus respuestas.

Criterio de terminado

- La secuencia se comporta como dice la tabla, y la medición y la prueba de falla están registradas.
- Test unitario del validador en verde.
- Commits locales hechos y publicados (cuando lo pidas) y **Calvo avisado**.

No tocar

La activación (`activate` y su endpoint), el modelo de la Parte 1 (salvo un cambio justificado y avisado), Postman, `CLAUDE.md`, el plan y los informes. **No** implementes `GET .../details` (decisión 11 sin confirmar).

Qué le dejás a Calvo

El endpoint de lecturas funcionando, los resultados de la medición de eficiencia y de la prueba de falla, y la lista de casos que ya probaste a mano (para convertirlos en pedidos de Postman).

Prompt para Claude Code (Parte 2)

```
Contexto: repo iw3-gas-terminal-backend (Spring Boot 4, Java 21, MySQL). Estamos en la Fase 3 (punto 3: activación de la carga y lecturas del caudalímetro), rama fase-3-carga-tiempo-real. El trabajo es SECUENCIAL: primero Gonzalez, después Contreras (esta parte) y después Calvo. El equipo autorizó la fase el 27/10/2026 y confirmó las decisiones D1 a D10 del documento de división, además de los commits locales atómicos. Las líneas de CLAUDE.md que dicen que la Fase 2 espera su PR o que la Fase 3 está pendiente quedaron desactualizadas: no las edites (las corrige Calvo al cerrar la fase).

Leé primero: CLAUDE.md, docs/plan-de-trabajo.md y docs/fase-3-division-de-trabajo.md (secciones 1 a 6 y "Parte 2"), y estudiá business/OrderBusiness.java (registerTare y activate), integration/loader/controllers/OrderLoaderRestController.java y model/OrderDetail.java.

Soy Joaquín Contreras y hago la PARTE 2 (lecturas del caudalímetro y descarte). Antes de empezar hacé git fetch, git checkout fase-3-carga-tiempo-real y git pull --rebase, y comprobá que Gonzalez (Parte 1) ya publicó: OrderDetail y OrderDetailRepository, los campos nuevos de Order, ForbiddenException, Constants.URL_INTEGRATION_LOADER y OrderLoaderRestController con la activación funcionando. Si falta algo, avisame y esperá.

Implementá SOLO estos archivos (rutas bajo src/main/java/ar/edu/iua/iw3/ salvo el test):
1. business/OrderDetailValidator.java: clase final de paquete, función pura static List<String> discardReasons(double accumulatedMass, double density, double flowRate, Double previousMass) que evalúa TODAS las condiciones y devuelve los motivos (vacía = válida): !(flowRate > 0) -> "el caudal (flowRate) debe ser mayor que 0"; !(accumulatedMass > 0) -> "la masa acumulada (accumulatedMass) debe ser mayor que 0"; previousMass != null && accumulatedMass < previousMass -> "la masa acumulada es menor que la anterior (<valor>)"; !(density > 0 && density < 1) -> "la densidad (density) debe estar entre 0 y 1, sin incluirlos". Usá la forma !(x > 0) para que NaN también se descarte. La temperatura no tiene regla. Una masa IGUAL a la anterior es válida.
2. src/test/java/ar/edu/iua/iw3/business/OrderDetailValidatorTest.java: JUnit 5 SIN @SpringBootTest: lectura válida, masa igual a la anterior, caudal 0 y negativo, masa 0 y negativa, masa menor que la anterior, densidad 0, 1, mayor que 1 y negativa, NaN y varios motivos juntos.
3. business/DetailResult.java: clase pública con saved (boolean) y reason (String), getters, y constructores estáticos saved() y discarded(String reason).
4. business/IOrderDetailBusiness.java: SOLO DetailResult add(Integer orderNumber, Double accumulatedMass, Double density, Double temperature, Double flowRate, LocalDateTime timestamp) throws NotFoundException, BadRequestException, ConflictException, BusinessException.
5. business/OrderDetailBusiness.java (@Service): inyecta IOrderBusiness (para load), OrderRepository y OrderDetailRepository. add con @Transactional(rollbackFor = Exception.class) y estas comprobaciones FUERA del try, en este orden: (a) load(orderNumber) -> 404; (b) algún Double nulo -> 400 con "accumulatedMass, density, temperature y flowRate son obligatorios"; (c) estado distinto de INITIAL_WEIGHING_REGISTERED o activationDateTime nulo -> 409 con "La orden <n> no acepta lecturas: debe estar en estado 2 y activada (estado <e>, activada=<true/false>)"; (d) motivos de descarte con el validador usando order.getLastAccumulatedMass() como masa anterior: si hay alguno, devolver DetailResult.discarded(motivos unidos con "; ") SIN escribir nada. Si es válida, DENTRO del try: estampa = timestamp o LocalDateTime.now() si es nulo; guardar un OrderDetail y actualizar la cabecera (lastAccumulatedMass, lastDensity, lastTemperature, lastFlowRate; loadingStartDateTime solo si es nulo; loadingEndDateTime siempre); devolver DetailResult.saved(). Cualquier otra falla -> log.error y BusinessException. No recorras el histórico para buscar la masa anterior.
6. integration/loader/model/DetailRequest.java (Double accumulatedMass, density, temperature, flowRate y LocalDateTime timestamp opcional) y DetailResponse.java (boolean saved, String reason), con @Schema y ejemplos. Usá Double y no double: un campo ausente debe dar 400, no 0.
7. integration/loader/controllers/OrderLoaderRestController.java: AGREGÁ la inyección de IOrderDetailBusiness y el endpoint POST /orders/{orderNumber}/details (consumes y produces JSON) con @Operation y @ApiResponse (200, 400, 404, 409). SIEMPRE 200 para lectura guardada ({saved:true, reason:null}) o descartada ({saved:false, reason:...}); 404, 400, 409 y 500 para los errores. NO modifiques el método de activación.

NO toques: la activación, el modelo de la Parte 1 (salvo un cambio justificado que me avises), el JSON de Postman, CLAUDE.md, docs/plan-de-trabajo.md ni los informes (Parte 3). NO implementes GET /orders/{n}/details (decisión 11 sin confirmar), ni la Fase 4 ni la 5. Si creés que hace falta un cambio ajeno, avisame en vez de hacerlo.

Reglas: metodología de la cátedra (capas, I*Business, excepciones checked, StandartResponse, Constants); identificadores en inglés y comentarios en español. Commits atómicos de 100 líneas o menos, en español, formato tipo(ámbito): descripción, SIN Co-Authored-By de Claude ni "Generated with Claude Code" (están autorizados los commits locales). No hagas push, merge ni force push salvo que yo lo pida; cuando lo pida, primero git pull --rebase. No agregues dependencias ni refactorices fuera de la fase. Antes de editar, mostrame tu plan de archivos y commits y esperá mi OK.

Verificá de verdad y reportá solo lo que ejecutaste: compilá; corré ./mvnw -B -Dtest=OrderDetailValidatorTest test; con la base levantada creá una orden, registrá la tara, activala y probá la secuencia: lectura 1 válida
```

```
(masa 100) y 2 válida (masa 250) con timestamps fijos y comprobá la cabecera (últimos valores, inicio = estampa
de la 1, fin = estampa de la 2); descartes (masa 200, caudal 0 y -5, masa 0 y -1, densidad 0, 1, 1.5 y -0.2) con
saved=false y su motivo, y que la cabecera NO cambió; masa igual (250) válida; temperatura negativa válida; sin
timestamp (el inicio no cambia y el fin queda con la hora del servidor); campo ausente, {}, JSON malformado,
"abc" y timestamp inválido (400); orden inexistente (404); orden en estado 1 y orden en estado 2 sin activar
(409); sin token (401). Comprobá con SELECT COUNT(*) FROM order_details WHERE id_order = <id> que solo están las
lecturas guardadas. Medí la eficiencia: 100 lecturas seguidas con masa creciente (un script corto) y anotá el
tiempo total y el promedio por lectura (orientativo: unos 100 ms o menos cada una en local). Probá qué responde
la app si la base falla al actualizar la cabecera (por ejemplo achicando TEMPORALMENTE una columna last_ de tu
base LOCAL y restaurándola después); si la respuesta no es un StandartResponse, evaluá saveAndFlush dentro del
try y dejá la conclusión escrita. Corré ./mvnw -B test y npx newman@6 run docs/postman/IW3-Gas-
Terminal.postman_collection.json (siguen pasando las 46 aserciones). Si algo falla, decilo.

Al terminar: agregá al final de docs/fase-3-division-de-trabajo.md el bloque "Estado de la Parte 2" (qué quedó
hecho, commits, resultados reales de las pruebas, la medición de eficiencia, la prueba de falla y la lista de
casos que Calvo debe pasar a Postman) y las filas de tus archivos en docs/guia-estudio-src.md marcadas F3
(commits aparte). No pushees hasta que yo lo pida.
```

Parte 3 — Pruebas, Postman, auditoría final y documentación

Responsable: Tomás Calvo · **Estimación:** ≈ 5 a 6 h · **Cierra la fase.**

Punto de partida (qué heredás de las Partes 1 y 2)

Antes de empezar, traé la rama y comprobá que están la activación y las lecturas funcionando, y leé los bloques "Estado de la Parte 1" y "Estado de la Parte 2" (con los casos probados, la medición de eficiencia y la prueba de falla). La carpeta 02 de Postman y las 46 aserciones son tu punto de comparación para las regresiones.

Objetivo

Probar la fase de punta a punta, **auditar si quedó bien hecha y si están todos los puntos cubiertos**, documentarla, corregir las inconsistencias de documentación que quedaron de antes y abrir el PR.

Qué hacer, paso a paso

Apartado A — Postman: carpeta `03 - Fase 3: carga (sistema de carga)`

Variables **propias**: `orderNumberLoad`, `activationPasswordLoad`, y las que necesites para las órdenes de los casos de estado. Estilo y generación del número de orden como en la carpeta 02 (`Date.now()` en el *pre-request script*). **No uses** `orderNumber` ni `activationPassword`.

Grupo	Pedidos (cada uno con sus tests)	Esperado
Preparación	Crear orden de carga · Tara de esa orden (guarda la contraseña) · Crear orden en estado 1 · Crear orden con tara sin activar	201 · 200 · 201 · 201 y 200
Activación	Sin campos · Sin `password` · Orden inexistente · Orden en estado 1 · Contraseña incorrecta · JSON malformado · Sin token	400 · 400 · 404 · 409 · **403** (y el cuerpo no incluye la contraseña) · 400 · 401
	Activación correcta · Consultar la orden · Activar de nuevo · Historial de estados	200 con `preset` 15000 · `activationDateTime` presente, estado 2, **sin** `activationPassword` y campos de carga en `null` · 200 y la misma `activationDateTime` · siguen **2** registros

Grupo	Pedidos (cada uno con sus tests)	Esperado
Lecturas válidas	Lectura 1 · Lectura 2 · Consultar la cabecera	`saved: true` · `saved: true` · últimos valores de la 2, inicio = estampa de la 1, fin = estampa de la 2
Descartes	Masa menor que la anterior · Caudal 0 · Caudal negativo · Masa 0 · Densidad 0 · Densidad 1 · Densidad 1.5 · Consultar la cabecera	`saved: false` con un fragmento del motivo (`anterior`, `caudal`, `masa`, `densidad`) · la cabecera **no cambió**
Casos límite	Masa igual a la anterior · Sin `timestamp`	`saved: true` · `saved: true`
Errores	Sin campo · JSON malformado · Valor no numérico · Orden no activada · Orden en estado 1 · Orden inexistente · Sin token	400 · 400 · 400 · 409 · 409 · 404 · 401

Asegurá que las aserciones sobre mensajes usen **fragmentos** (`include`), según la sección 3.5. Usá timestamps fijos sin zona para poder comparar el inicio y el fin de carga.

El JSON de Postman va en **un commit aparte**, solo ese archivo, y con `git diff` verificás que **solo cambie lo nuevo**.

Apartado B — Prueba de punta a punta

```
npx newman@6 run docs/postman/IW3-Gas-Terminal.postman_collection.json
./mvnw -B test        # con la base arriba
```

Registrá los comandos y los resultados **reales** (pedidos, aserciones y fallas). Tienen que seguir pasando las carpetas 01 y 02 (46 aserciones). Para el descarte, además, usá `SELECT COUNT(*) FROM order_details WHERE id_order = <id>` y compará con las lecturas guardadas.

Apartado C — Auditoría final: ¿quedó bien hecha la Fase 3?

Esta tarea es **adicional** y es la más importante de tu parte. Armá una **matriz de trazabilidad** en el informe: cada requisito de la consigna y del plan, dónde está implementado (archivo y método), con qué prueba se verificó y el resultado. Clasificá cada fila como **implementado y verificado**, **implementado y no verificado**, **parcial**, **con defectos** o **pendiente**.

Revisá como mínimo:

1. **Los 19 criterios de la sección 6**, uno por uno, con evidencia (pedido, respuesta o consulta SQL).
2. **Las decisiones D1 a D10**: que el código haga exactamente lo decidido (por ejemplo, que la activación no escriba en el historial, que la masa igual sea válida, que `timestamp` ausente use la hora del servidor).
3. **El orden de las comprobaciones** (D7) y que ninguna quede dentro del `try`.
4. **Que no haya lógica duplicada**, ni clases de la Fase 4 o 5, ni dependencias nuevas, ni una lista de detalles en `Order`.
5. **Que la contraseña no aparezca** en ningún mensaje, log ni respuesta de consulta.
6. **La consigna original en PDF**: comparar los nombres de campos del JSON de las lecturas y las reglas de descarte con lo implementado. Si el PDF no está a mano, dejarlo anotado como "no verificado contra el original".
7. **La preparación para la Fase 4**: que un cambio de estado a 3 haga que las lecturas se rechacen con 409 sin modificar el código de la Fase 3.
8. **Regresiones** de las Fases 1 y 2.

Si encontrás un defecto: lo registrás con evidencia (pedido exacto, respuesta obtenida, archivo y método) y se lo pasás a su dueño. Si es un cambio mínimo y el dueño lo autoriza, lo corregís vos en un commit `fix(...)` aparte, explicando por qué. **No cierres la fase como completa** si queda un criterio esencial sin cumplir o sin verificar: indicá "parcialmente completa" y qué falta.

Apartado D — Documentación

1. `docs/fase-3-carga-tiempo-real.md` con el formato de `docs/fase-2-pesaje-inicial.md`:
   - **Informe técnico:** alcance y resultado, decisiones de diseño (D1 a D10 con la alternativa descartada), inventario de archivos por parte (archivo, acción, motivo, responsabilidad), recorrido de las dos peticiones, funciones y métodos, pruebas con resultados reales, **cambios de comportamiento respecto de la Fase 2**, hallazgos y limitaciones, y la **matriz de auditoría** del apartado C.
   - **Guía de estudio A–J** con **10 preguntas o más** y respuestas modelo, y una sección "qué debe poder explicar cada integrante" (Gonzalez, Contreras y Calvo).
   - La atribución es **documental**: distinguí entre responsabilidad asignada y cambios observados en los commits.
2. **Limitaciones a registrar** (continuando la numeración H-12 y H-13): concurrencia en lecturas simultáneas, contraseña de 5 dígitos sin límite de intentos, vencimiento del token en cargas largas, zona horaria descartada si el `timestamp` trae `Z`, y lo que muestren la medición de eficiencia y la prueba de falla de la Parte 2.
3. **Control de calidad:** todo lo documentado coincide con el código real (nombres de clases y métodos, rutas, códigos, ejemplos). Si hay una discrepancia, se corrige el **documento**, no el código.

Apartado E — Corregir las inconsistencias de documentación (autorizado por Martín)

Estas correcciones se hacen **al cerrar la Fase 3**, con el estado real ya verificado y con cambios mínimos, en commits aparte:

Documento	Qué corregir
`CLAUDE.md` sección 2	"Última actualización", "Última fase completada" (Fase 3), "Qué sigue" (Fase 4), la fila de `main` (hoy dice que solo tiene las Fases 0 y 1 y que la Fase 2 espera su PR), la tabla de fases, y la fila de despliegue
`CLAUDE.md` secciones 5 y 8.1	Agregar la Fase 3 en "Qué se hizo"; la Fase 2 figura como integrada (PR #6)
`CLAUDE.md` sección 8.3	**Quitar "Confirmar el código HTTP de estado incorrecto"** (ya es 409). Dejar actualizado el estado de la protección de `main` y el redespliegue
`CLAUDE.md` secciones 6, 7 y 9	Paquetes `integration/tms` y `integration/loader` y las clases nuevas; los endpoints de la tara y de la Fase 3; las entidades `Order` (campos nuevos) y `OrderDetail`; las decisiones 18 a 27
`CLAUDE.md` sección 10	Hallazgos nuevos de la Fase 3
`CLAUDE.md` sección 14	Sumar `docs/fase-2-pesaje-inicial.md`, `docs/fase-3-carga-tiempo-real.md`, `docs/guia-estudio-src.md` y las guías de división
`docs/fase-2-pesaje-inicial.md`	Los dos tests se llaman `generatesFiveDigitPasswords` y `doesNotAlwaysReturnTheSameValue` (el informe dice `testRangeAndFormat` y `testEntropy`), y la base de las pruebas es el contenedor `iw3-gas-mysql` (el informe dice `iw3-mysql`)
`docs/plan-de-trabajo.md`	Decisiones 18 a 27 y el estado de la Fase 3 (solo después de verificarla)
`docs/guia-estudio-src.md`	Archivos nuevos (si faltan filas), tabla de endpoints y fecha de actualización
PDF	Regenerar los de la Fase 3, de la guía de `src`, de la Fase 2 (corregida) y de esta división

No se modifica nada de `CLAUDE.md` ni del plan fuera de esta lista. Los puntos de configuración del repositorio (el ruleset todavía pide **1** aprobación en lugar de las 3 acordadas, y no borra las ramas al mergear) los cambia el dueño (`joacoContreras`): anotalos en los pendientes y avisale.

Apartado F — Pull Request

Abrí el PR como **borrador**, con la descripción de los cambios por parte y cómo se probó. **Benjamín**, Gonzalez y Contreras lo revisan y aprueban (tu aprobación no cuenta). Después del merge: `git checkout main` y `git pull`, y falta **redesplegar la VM** (que sigue sin la Fase 2 ni la 3).

Apartado G — Opcional, solo si Martín aprueba la decisión 11

`GET /api/v1/orders/{orderNumber}/details`, paginado (`page`, `size` por defecto 20 y máximo 100, orden fijo por `id` ascendente, ignorando el `sort` del cliente, respuesta con `PagedModel`; 404 si la orden no existe). Implica `list` en `IOrderDetailBusiness` y `OrderDetailBusiness`, `findByOrder(Order, Pageable)` en `OrderDetailRepository` y el método en `OrderRestController`. **No lo hagas sin la aprobación.**

Criterio de terminado

- Carpeta 03 de Postman y las carpetas 01 y 02 pasan con newman (resultados reales registrados).
- Auditoría con la matriz completa y los defectos (si hay) registrados.
- Informe, guía, correcciones del apartado E y PDF hechos y revisados contra el código.
- PR abierto como borrador, sin `.postman/` ni `postman/`.

Prompt para Claude Code (Parte 3)

```
Contexto: repo iw3-gas-terminal-backend (Spring Boot 4, Java 21, MySQL). Estamos en la Fase 3 (punto 3: activación de la carga y lecturas del caudalímetro), rama fase-3-carga-tiempo-real. El trabajo es SECUENCIAL: primero Gonzalez, después Contreras y después Calvo (esta parte, que cierra la fase). El equipo autorizó la fase el 27/10/2026 y confirmó las decisiones D1 a D10 del documento de división, además de los commits locales atómicos y la corrección de las inconsistencias de documentación al cerrar la fase 3.

Leé primero: CLAUDE.md, docs/plan-de-trabajo.md, docs/fase-2-pesaje-inicial.md (como molde del informe y la guía) y docs/fase-3-division-de-trabajo.md completo (en especial las secciones 3, 6 y "Parte 3" y los bloques "Estado de la Parte 1" y "Estado de la Parte 2"). Revisá también la colección docs/postman/IW3-Gas-Terminal.postman_collection.json para imitar su estilo.

Soy Tomás Calvo y hago la PARTE 3 (pruebas, Postman, auditoría final, documentación y PR). Antes de empezar hacé git fetch, git checkout fase-3-carga-tiempo-real y git pull --rebase, y comprobá que Gonzalez (Parte 1) y Contreras (Parte 2) ya publicaron la activación y las lecturas. Si falta algo, avisame y esperá.

Tu trabajo, en este orden:
1. Postman: agregá a la colección una carpeta "03 - Fase 3: carga (sistema de carga)" con VARIABLES PROPIAS (orderNumberLoad, activationPasswordLoad y las que necesites; NO uses orderNumber ni activationPassword). Cubrí: preparación (crear orden, tara con la contraseña guardada, orden en estado 1, orden con tara sin activar); activación (sin campos 400, sin password 400, inexistente 404, estado 1 409, contraseña incorrecta 403 sin la contraseña en el cuerpo, JSON malformado 400, sin token 401, correcta 200 con preset, consultar la orden con activationDateTime presente y SIN activationPassword, repetir la activación 200 con la misma activationDateTime, historial con 2 registros); lecturas válidas (dos lecturas y la cabecera con últimos valores, inicio y fin); descartes (masa menor que la anterior, caudal 0, caudal negativo, masa 0, densidad 0, 1 y 1.5, y la cabecera sin cambios); límite (masa igual a la anterior válida y sin timestamp); errores (campo ausente, JSON malformado, valor no numérico, orden no activada 409, orden en estado 1 409, inexistente 404, sin token 401). Las aserciones sobre mensajes con fragmentos (include), según la sección 3.5. Timestamps fijos sin zona. No modifiques los pedidos de las Fases 1 y 2. Commit del JSON aparte, solo ese archivo, y verificá con git diff que solo cambie lo nuevo.
2. Corré todo: npx newman@6 run docs/postman/IW3-Gas-Terminal.postman_collection.json y ./mvnw -B test (con la base arriba). Tienen que seguir pasando las 46 aserciones de las Fases 1 y 2. Comprobá el descarte también con SELECT COUNT(*) FROM order_details WHERE id_order = <id>. Registrá comandos y resultados REALES.
3. AUDITORÍA FINAL: auditá si la Fase 3 quedó bien hecha y si están todos los puntos cubiertos. Armá una matriz de trazabilidad (requisito de la consigna y del plan -> archivo y método -> prueba -> resultado -> clasificación: implementado y verificado / implementado y no verificado / parcial / con defectos / pendiente). Revisá: los 19 criterios de la sección 6 con evidencia; que el código haga exactamente lo decidido en D1 a D10; el orden de comprobaciones y que ninguna esté dentro del try; que no haya lógica duplicada, ni código de la Fase 4 o 5, ni dependencias nuevas, ni una lista de detalles en Order; que la contraseña no aparezca en mensajes, logs ni consultas; la comparación con la consigna original en PDF si está disponible (si no, dejalo anotado como no verificado contra el original); que un cambio a estado 3 haría rechazar las lecturas con 409 sin tocar la Fase 3; y las regresiones de las Fases 1 y 2. Si encontrás un defecto, no lo arregles de entrada: registralo con el pedido exacto, la respuesta, el archivo y el método, y decime a qué parte corresponde para avisarle a su dueño; si es un cambio mínimo y el dueño lo autoriza, hacelo en un commit fix aparte explicando el motivo. No des la fase por completa si queda un criterio esencial sin cumplir o sin verificar: decí que está parcialmente completa y qué falta.
4. Documentación: creá docs/fase-3-carga-tiempo-real.md con el formato de docs/fase-2-pesaje-inicial.md (informe técnico con inventario de archivos por parte, decisiones D1 a D10, recorridos de las dos peticiones, funciones y métodos, pruebas con resultados reales, cambios de comportamiento respecto de la Fase 2, hallazgos y limitaciones —continuá la numeración H-12 y H-13— y la matriz de auditoría; y guía de estudio A-J con 10 preguntas o más y una sección de qué debe poder explicar cada integrante), integrando los bloques de las Partes 1 y 2. La atribución es documental: distinguí responsabilidad asignada de cambios observados. Controlá que todo lo escrito coincida con el código real; si no coincide, corregí el documento y no el código.
5. Corregí las inconsistencias de documentación (autorizado), en commits aparte y con cambios mínimos: CLAUDE.md secciones 2, 5, 6, 7, 8.1, 8.3, 9, 10 y 14 (estado real tras la Fase 3, la Fase 2 integrada por el PR #6, quitar "Confirmar el código HTTP de estado incorrecto" porque ya es 409, endpoints, entidades, paquetes, decisiones 18 a 27 y mapa de documentos); docs/fase-2-pesaje-inicial.md (los tests son generatesFiveDigitPasswords y doesNotAlwaysReturnTheSameValue, y la base de las pruebas es el contenedor iw3-gas-mysql); docs/plan-de-trabajo.md (decisiones 18 a 27 y estado de la Fase 3 solo después de verificarla); docs/guia-estudio-src.md (filas, endpoints y fecha). No modifiques nada más de CLAUDE.md ni del plan. Regenerá los PDF (ver docs/pdf/README.md).
6. Opcional SOLO si Martín aprobó la decisión 11: GET /api/v1/orders/{orderNumber}/details paginado. Si no está aprobada, NO lo hagas.
7. Abrí el PR como borrador solo cuando yo te lo pida, sin las carpetas .postman/ ni postman/.

NO toques el código de src/main ni src/test salvo un fix mínimo y autorizado de la auditoría. No implementes la Fase 4 ni la 5. Los problemas de configuración del repositorio (el ruleset pide 1 aprobación en vez de 3 y no
```

```
borra las ramas al mergear) los cambia el dueño: anotalos como pendientes.

Reglas: documentación en español, identificadores en inglés. Commits atómicos de 100 líneas o menos (excepción:
documentos completos nuevos y el JSON de Postman, un commit aparte cada uno), en español, formato tipo(ámbito):
descripción, SIN Co-Authored-By de Claude ni "Generated with Claude Code" (están autorizados los commits
locales). No hagas push, merge ni force push salvo que yo lo pida; cuando lo pida, primero git pull --rebase.
Reportá solo lo que ejecutaste; si algo falla, decilo. Antes de editar, mostrame tu plan y esperá mi OK.
```

Cierre de la fase

Checklist de integración

✓	Verificación	Quién
☐	`./mvnw -B -DskipTests compile` sin errores en la rama	Todos
☐	`./mvnw -B test` pasa (con la base arriba)	Gonzalez y Calvo
☐	La app arranca contra una base que ya tenía órdenes	Gonzalez
☐	Los 19 criterios de la sección 6, con evidencia	Calvo
☐	newman: carpeta 03 y carpetas 01 y 02 en verde (46 aserciones + las nuevas)	Calvo
☐	Medición de eficiencia y prueba de falla registradas	Contreras
☐	Auditoría final con matriz completa y defectos registrados	Calvo
☐	Correcciones de documentación del apartado E hechas y PDF regenerados	Calvo
☐	Revisión cruzada (Gonzalez → Contreras → Calvo → Gonzalez)	Los tres
☐	PR abierto como borrador, sin `.postman/` ni `postman/`	Calvo
☐	Aprobaciones de Gonzalez, Contreras y Benjamín antes de mergear	Equipo
☐	Después del merge: reconstruir la imagen y redesplegar la VM	Equipo
☐	Pendiente del dueño del repo: el ruleset a 3 aprobaciones y borrado de ramas al mergear	`joacoContreras`

Problemas frecuentes

Síntoma	Causa probable	Qué hacer
El push es rechazado	Alguien pusheó antes	`git pull --rebase` y volver a pushear
No compila: no encuentra `ForbiddenException`, `OrderDetail` o `DetailResult`	Falta traer el push de la parte anterior	`git pull --rebase`
500 en vez de 403, 409, 404 o 400	La comprobación quedó dentro del `try/catch`	Moverla fuera del `try`
Un campo ausente se descarta en lugar de dar 400	El pedido usa `double` y no `Double`	Usar `Double`
Error de columna al arrancar	Un campo nuevo de `Order` no es opcional	Usar `Double` / `LocalDateTime` sin `nullable = false`
`GET /orders/{n}` devuelve una respuesta enorme	`Order` tiene una lista de detalles	Quitarla: el detalle solo apunta a la orden
Falla una aserción de la Fase 1 o 2 en newman	Se usó `orderNumber` o `activationPassword` en la carpeta 03	Usar variables propias
401 en todo	Token vencido (dura 1 hora) o falta el usuario de prueba	Volver a loguearse / correr el seed

Síntoma	Causa probable	Qué hacer
El contenedor de la base no arranca	Docker apagado	Iniciar Docker; el contenedor del TP es `iw3-gas-mysql`

Después de la Fase 3

Siguen la **Fase 4** (cerrar la orden: estado 3, y rechazar las lecturas que lleguen después) y la **Fase 5** (pesaje final, conciliación con promedios y el circuito completo). Las dos se apoyan en lo que se deja acá: el estado 2 con la orden activada, la cabecera con el producto cargado (`lastAccumulatedMass`) y el histórico en `order_details`.

Anexo A — Hallazgos del diagnóstico previo

Registrados al preparar esta fase, para que no se pierdan:

#	Hallazgo	Qué se hace
1	`CLAUDE.md` quedó desactualizado: sigue diciendo que `main` tiene solo las Fases 0 y 1 y que la Fase 2 espera su PR; mantiene "confirmar el código de estado incorrecto" (ya es 409); su mapa de documentos no lista los de la Fase 2	Lo corrige Calvo al cerrar la Fase 3 (apartado E de la Parte 3)
2	El informe de la Fase 2 nombra tests que no existen (`testRangeAndFormat`, `testEntropy`) y dice que corrió contra `iw3-mysql` (el de la cátedra; el del TP es `iw3-gas-mysql`)	Lo corrige Calvo (apartado E de la Parte 3)
3	El ruleset de `main` exige **1** aprobación, no las 3 acordadas, y no borra las ramas al mergear	Lo cambia el dueño del repo (`joacoContreras`)
4	La consigna original en PDF no está en las máquinas del equipo; solo existe su resumen en `CLAUDE.md` y el plan	Calvo lo contrasta si consigue el PDF (apartado C de la Parte 3)
5	`.postman/` y `postman/` aparecen como carpetas sin seguimiento y no están en `.gitignore`	No se commitean
6	La VM sigue con la imagen de la Fase 1	Se redesplegará después del merge
7	En una prueba descartable previa (fuera del repo, ya eliminada) un `timestamp` con `Z` se aceptó descartando la zona; no se llegó a medir la eficiencia ni la falla al confirmar la transacción	Quedan como trampas 10, 11 y 12 de la sección 4, a verificar en la Parte 2

Anexo B — Registro de traspasos

Cada responsable completa su fila al terminar y **agrega debajo su bloque "Estado de la Parte N"**.

Parte	Responsable	Estado	Último commit	Fecha
1	Martín González	Completa, verificada y mergeada (PR #7)	`b090dfa` (último commit de código)	27/10/2026
2	Joaquín Contreras	Completa, verificada y mergeada (PR #7)	`5971075` (último commit de código)	27/10/2026
3	Tomás Calvo	Completa y verificada (incluye el arreglo de H-14 y la decisión 11)	`041308a` (último commit de código y Postman)	27/10/2026

Modelo del bloque "Estado de la Parte N"

```
## Estado de la Parte N (dd/mm/aaaa)

| Paso | Commit | Estado |
|---|---|---|
| ... | `abc1234` `tipo(ámbito): descripción` | Pusheado / Local |

**Qué quedó implementado** (archivos y qué hace cada uno).

**Verificado** (comandos y resultados reales; marcá lo que NO pudiste ejecutar).

**Para tener en cuenta** (lo que necesita saber el siguiente: limitaciones, decisiones, casos probados,
mediciones).
```

Estado de la Parte 1 (27/10/2026)

Paso	Commit	Estado
Rama y documentación (división y PDF)	`1578e5c` `docs(fase-3): agrega la división del trabajo y los prompts de las 3 partes`	Local
Índice de PDF	`ddc3361` `docs(pdf): agrega el PDF de la división de la Fase 3 al índice`	Local
Campos de la orden	`edceb00` `feat(model): agrega a la orden los campos de activación y de carga`	Local
`OrderDetail` y su repositorio	`0ff6542` `feat(model): agrega la entidad OrderDetail y su repositorio`	Local
`ForbiddenException`	`f6b8576` `feat(business): agrega ForbiddenException para la contraseña incorrecta`	Local
URL del sistema de carga	`a1c3eb2` `feat(controllers): agrega la URL de integración del sistema de carga`	Local
Regla `activate`	`cca066e` `feat(business): agrega la activación de la orden`	Local
Pedido y respuesta de la activación	`61f65d6` `feat(integration): agrega el pedido y la respuesta de la activación`	Local
Endpoint de activación	`b090dfa` `feat(integration): agrega el endpoint de activación del sistema de carga`	Local

Los commits de documentación de esta parte son `98357e5` (este bloque) y `4362b25` (filas de la guía de `src`). **No hay push todavía:** la rama `fase-3-carga-tiempo-real` existe solo en la máquina de Gonzalez. El PDF de este documento no se regeneró (lo hace Calvo al cerrar, apartado E de la Parte 3).

Qué quedó implementado (rutas bajo `src/main/java/ar/edu/iua/iw3/`):

- `model/Order.java`: siete campos opcionales con `@Schema(READ_ONLY)`: `activationDateTime`, `loadingStartDateTime`, `loadingEndDateTime` (`LocalDateTime`) y `lastAccumulatedMass`, `lastDensity`, `lastTemperature`, `lastFlowRate` (`Double`). No tiene lista de detalles.
- `model/OrderDetail.java`: entidad de la tabla `order_details`. `order` es `@ManyToOne(optional = false, fetch = LAZY)` con `@JsonIgnore` y columna `id_order`; cuatro `double` obligatorios y `timestamp` en la columna `reading_timestamp`. Tiene `@AllArgsConstructor`: `new OrderDetail(Long id, Order order, double accumulatedMass, double density, double temperature, double flowRate, LocalDateTime timestamp)`.
- `model/persistence/OrderDetailRepository.java`: `JpaRepository<OrderDetail, Long>` sin métodos propios.
- `business/exception/ForbiddenException.java`: se traduce a 403 (mismo molde que `ConflictException`).
- `controllers/Constants.java`: `URL_INTEGRATION_LOADER`.
- `business/IOrderBusiness.java` y `OrderBusiness.java`: `activate(orderNumber, password)` con `@Transactional(rollbackFor = Exception.class)` y las comprobaciones fuera del `try`: cuerpo incompleto

(400) → orden inexistente (404) → estado distinto de 2 (409) → contraseña incorrecta (403). Si `activationDateTime` es nulo lo guarda con la hora actual; si ya estaba, no lo toca (idempotente). No escribe en el historial de estados.

- `integration/loader/model/ActivationRequest.java` (`Integer orderNumber`, `Integer password`) y `ActivationResponse.java` (`Double preset`).
- `integration/loader/controllers/OrderLoaderRestController.java`: **solo** `POST /api/v1/integration/loader/activation`. Inyecta `IOrderBusiness` y `IStandartResponseBusiness`.

Verificado (resultados reales del 27/10/2026, contra MySQL real y la base `iw3_db`):

Verificación	Resultado
`./mvnw -B -q -DskipTests compile` tras cada commit de código	OK
`./mvnw -B test`	3 tests (los 2 del generador y `contextLoads`), 0 fallas, `BUILD SUCCESS`
Esquema	Hibernate creó `order_details` y agregó las 7 columnas, todas opcionales, sobre una base con 4 órdenes. Las 4 órdenes siguieron intactas, con los campos nuevos en `NULL`
Arranque de la app	`Started Iw3Application in 10.4 s` y 0 `ERROR` en el log
Activación (script temporal de 34 comprobaciones contra `http://localhost:8080`, ya eliminado)	34 PASS, 0 FAIL (detalle abajo)
Cruce con la base	La orden activada quedó en estado 2 con `activation_date_time` guardado; su historial tiene 2 registros (`null→1` y `1→2`); `order_details` tiene 0 filas
Swagger (`/v3/api-docs`)	El path `POST /api/v1/integration/loader/activation` figura con las respuestas 200, 400, 403, 404 y 409, el tag "Integración sistema de carga" y los esquemas `ActivationRequest` y `ActivationResponse` con ejemplos. El esquema `Order` incluye los 7 campos nuevos y no incluye `activationPassword`
Regresión: `npx newman@6 run docs/postman/IW3-Gas-Terminal.postman_collection.json`	21 pedidos, **46 aserciones, 0 fallas** (2,6 s)

Casos de activación comprobados: `{}`, sin `password`, sin `orderNumber`, JSON malformado, `password` `"abc"` y `orderNumber` `"abc"` → **400**; orden inexistente → **404**; orden en estado 1 → **409** (`La orden <n> no está lista para activar (estado 1)`); contraseña incorrecta → **403** (la orden no queda activada y el cuerpo no incluye ninguna contraseña); sin token → **401**; contraseña correcta → **200** `{ "preset": 15000.0 }` con `activationDateTime` guardado y estado 2; repetir la activación → **200**, mismo preset y **misma** `activationDateTime`; orden ya activada con contraseña incorrecta → **403** sin alterar la fecha; `/status-log` sigue con 2 registros. `GET /orders/{n}` muestra los campos nuevos (en `null` hasta que haya lecturas), no muestra `activationPassword` y no trae lista de detalles.

No ejecutado o no verificado: Swagger UI en el navegador (solo el JSON de `/v3/api-docs`); órdenes en estado 3 o 4 (ninguna operación las crea todavía); activaciones simultáneas, rendimiento y la falla de base al confirmar la transacción (trampa 12); la carpeta 03 de Postman (es de la Parte 3).

Limpieza: los datos de prueba (órdenes 18 a 21, las de mi script y las de newman, y las entidades con códigos `P1` y `TMS-01`) se borraron y la base quedó con los mismos datos que antes (4 órdenes). Permanecen la tabla `order_details` (vacía) y las 7 columnas nuevas, que agrega Hibernate al arrancar.

Para tener en cuenta (Contreras):

- **Qué recibís:** `OrderDetail` con su constructor completo, `OrderDetailRepository` (vacío), los 7 campos de `Order` con sus getters y setters de Lombok (los últimos valores son `Double`, o sea, pueden ser nulos) y `OrderLoaderRestController` con la activación funcionando. En ese controller solo tenés que **agregar** la

inyección de `IOrderDetailBusiness` y el método de lecturas, sin tocar `activate`. Ya están importados `BadRequestException`, `BusinessException`, `ConflictException`, `ForbiddenException`, `NotFoundException`, `Content`, `Schema` y `ApiResponse`.

- **Para probar necesitás una orden activada:** crearla (`POST /integration/sap/orders`), registrar la tara (`POST /integration/tms/orders/{n}/tare`, que devuelve la contraseña) y llamar a `POST /integration/loader/activation`.
- **Hora de Argentina:** `activationDateTime` se guarda con la zona configurada (`hibernate.jdbc.time_zone`). Vale lo mismo para las estampas que guardes.
- **Observaciones de Jackson:** un pedido sin `Content-Type` responde **415**, y un número enviado como texto (`"password": "26566"`) **se acepta** porque Jackson lo convierte. Lo mismo va a pasar con `"accumulatedMass": "250"` en las lecturas; un texto no numérico (`"abc"`) sí da 400.
- **Trampa 12:** `activate` usa `save` igual que `registerTare`, así que la actualización se envía a la base al confirmar la transacción, fuera del `try`. Tu verificación de la falla al confirmar aplica a las dos operaciones.
- **Limitaciones de la activación** (para registrar en la Parte 3): dos activaciones simultáneas de la misma orden podrían guardar la fecha dos veces (gana la última) y no hay límite de intentos de contraseña.
- **Si tu base ya tiene órdenes:** Hibernate agrega `order_details` y las columnas al arrancar, sin tocar los datos existentes.

Para tener en cuenta (Calvo):

- **La guía de `src` ya tenía filas duplicadas** en la sección 15 ("Buscador rápido"): las filas desde "Cambiar un código HTTP" hasta "Cambiar la conexión a la base" aparecen dos veces (vienen de la Fase 2). No las toqué; corregilas al cerrar la fase (apartado E).
- **En la guía solo agregué las filas de mis archivos** (marcadas F3) y el paso a paso de `activate`. La tabla de endpoints (sección 11), la tabla de fases (sección 2), la lista de códigos HTTP (falta el 403) y la fecha de actualización son tuyas.
- **Ni el PDF de la guía ni el de este documento se regeneraron** en la Parte 1.

Estado de la Parte 2 (27/10/2026)

Paso	Commit	Estado
Reglas de descarte	`0e84cb8` `feat(business): agrega las reglas de descarte de las lecturas`	Local
Pruebas unitarias de descarte	`d5ed889` `test(business): prueba las reglas de descarte`	Local
DTO de resultado	`6eb915c` `feat(business): agrega DetailResult`	Local
Interfaz de negocio de lecturas	`f324515` `feat(business): agrega la interfaz del negocio de lecturas`	Local
Lógica de recepción y cabecera	`85fc7cf` `feat(business): implementa la recepción de lecturas y la actualización de la cabecera`	Local
DTOs de pedido y respuesta	`7540a79` `feat(integration): agrega el pedido y la respuesta de las lecturas`	Local
Endpoint de lecturas	`3861b3e` `feat(integration): agrega el endpoint de lecturas del caudalímetro`	Local
Ajuste transaccional (trampa 12)	`5971075` `fix(business): usa saveAndFlush para capturar errores de base dentro del try`	Local

Qué quedó implementado:

- `business/OrderDetailValidator.java`: clase final de paquete con la función pura estática `discardReasons(accumulatedMass, density, flowRate, previousMass)`. Evalúa todas las condiciones sin

cortar y devuelve la lista de motivos (vacía si es válida). Usa la forma `!(x > 0)` para descartar `NaN`. La temperatura no tiene regla. Una masa igual a la anterior es válida.

- `src/test/java/ar/edu/iua/iw3/business/OrderDetailValidatorTest.java`: 8 pruebas unitarias con JUnit 5 puro (sin `@SpringBootTest`) cubriendo lectura válida, masa igual, caudal 0 y negativo, masa 0 y negativa, masa menor que la anterior, densidades inválidas (0, 1, 1.5, -0.2), `NaN` en los tres campos y acumulación de múltiples motivos.
- `business/DetailResult.java`: resultado de negocio con `saved` (boolean), `reason` (String), getters y constructores estáticos `saved()` y `discarded(reason)`.
- `business/IOrderDetailBusiness.java`: interfaz con el método `DetailResult add(...)`.
- `business/OrderDetailBusiness.java` (`@Service`): inyecta `IOrderBusiness`, `OrderRepository` y `OrderDetailRepository`. `@Transactional(rollbackFor = Exception.class)` con comprobaciones fuera del `try`: orden inexistente (404) → campos nulos (400) → estado distinto de 2 o no activada (409) → descarte (devuelve `DetailResult.discarded(...)` con motivos unidos por `"; "` sin escribir nada). Si es válida, dentro del `try`: guarda `OrderDetail`, actualiza cabecera de `Order` (`lastAccumulatedMass`, `lastDensity`, `lastTemperature`, `lastFlowRate`, `loadingStartDateTime` si es null y `loadingEndDateTime` siempre) usando `saveAndFlush(order)` y devuelve `DetailResult.saved()`.
- `integration/loader/model/DetailRequest.java`: DTO con `Double` para los 4 valores numéricos y `LocalDateTime timestamp` opcional, con anotaciones `@Schema` y ejemplos.
- `integration/loader/model/DetailResponse.java`: DTO con `boolean saved` y `String reason`, documentado con `@Schema`.
- `integration/loader/controllers/OrderLoaderRestController.java`: agrega la inyección de `IOrderDetailBusiness` y el endpoint `POST /orders/{orderNumber}/details`. Responde siempre 200 con `DetailResponse` (guardada o descartada), y traduce excepciones con `IStandartResponseBusiness`: 400, 404, 409 y 500. El método de activación no fue modificado.

Verificado (resultados reales del 27/10/2026 contra MySQL real `iw3_db`):

Verificación	Resultado
`./mvnw -B -Dtest=OrderDetailValidatorTest test`	8 tests ejecutados, 0 fallas (0.108 s), `BUILD SUCCESS`
`./mvnw -B test`	11 tests ejecutados (8 validator + 2 generator + 1 contextLoads), 0 fallas (11.2 s), `BUILD SUCCESS`
Swagger (`/v3/api-docs`)	`POST /orders/{orderNumber}/details` documentado con tag "Integración sistema de carga", esquemas `DetailRequest` y `DetailResponse`, y respuestas 200, 400, 404 y 409
Pruebas de secuencia sobre orden creada y activada	Todas PASS (14/14 casos): lectura 1 (100 kg), lectura 2 (250 kg), 9 descartes, masa igual (250 kg), temp negativa (-8.5 °C), sin timestamp, 5 errores sintácticos (400), inexistente (404), estado 1 (409), estado 2 sin activar (409), sin token (401)
Cruce con la base tras descartes	Cabecera intacta tras descartes (masa en 250 kg y fin en estampa de la 2); `SELECT COUNT(*) FROM order_details` arrojó exactamente 5 registros guardados
Medición de eficiencia (100 lecturas seguidas)	100 peticiones HTTP con masa creciente completadas en **3.61 s** (**36.1 ms por lectura** promedio en local). El total en base pasó de 5 a 105
Falla al confirmar la cabecera (trampa 12)	Se achicó temporalmente la columna `last_flow_rate` a `VARCHAR(2)`: con `save(order)` Spring respondía el JSON de error 500 por defecto (`{timestamp, status, error, trace, path}`). Con `saveAndFlush(order)` el flush ocurre dentro del `try`, la excepción es capturada y el controller responde con `StandartResponse` 500 (`{message, code: 500, devInfo}`). Se restauró la columna a `DOUBLE` y se adoptó `saveAndFlush`
Regresión Postman (`npx newman@6 run ...`)	21 pedidos, **46 aserciones, 0 fallas** (1.4 s)

Casos comprobados que Calvo debe pasar a Postman (carpeta 03):

1. `Lectura válida 1 (inicio de carga)`: 200 `{ "saved": true, "reason": null }`; cabecera con inicio y fin iguales al timestamp provisto.
2. `Lectura válida 2 (avance de carga)`: 200 `{ "saved": true, "reason": null }`; cabecera con inicio = ts1, fin = ts2 y últimos valores actualizados.
3. `Descarte: masa menor que la anterior (200 < 250)`: 200 `{ "saved": false, "reason": "la masa acumulada es menor que la anterior (250.0)" }`.
4. `Descarte: caudal 0 y negativo`: 200 `{ "saved": false, "reason": "el caudal (flowRate) debe ser mayor que 0" }`.
5. `Descarte: masa 0 y negativa`: 200 `{ "saved": false, "reason": "la masa acumulada (accumulatedMass) debe ser mayor que 0..." }`.
6. `Descarte: densidad fuera de (0, 1)` (0, 1, 1.5, -0.2): 200 `{ "saved": false, "reason": "la densidad (density) debe estar entre 0 y 1, sin incluirlos" }`.
7. `Lectura válida: masa igual a la anterior (250.0)`: 200 `{ "saved": true, "reason": null }`.
8. `Lectura válida: temperatura negativa (-8.5 °C)`: 200 `{ "saved": true, "reason": null }`.
9. `Lectura válida sin timestamp`: 200 `{ "saved": true, "reason": null }`; inicio intacto y fin con hora del servidor.
10. `Errores sintácticos (400)`: campo numérico ausente, `{}` vacío, JSON malformado, texto `"abc"` en número, `timestamp` con formato inválido.
11. `Orden inexistente`: 404 (`No se encuentra la Orden numero=<n>`).
12. `Orden en estado 1`: 409 (`La orden <n> no acepta lecturas: debe estar en estado 2 y activada (estado 1, activada=false)`).
13. `Orden en estado 2 sin activar`: 409 (`La orden <n> no acepta lecturas: debe estar en estado 2 y activada (estado 2, activada=false)`).
14. `Sin token`: 401.

Limpieza: se eliminaron la orden 92001 de pruebas y sus detalles; la tabla `order_details` quedó en 0 filas y la base con los datos limpios.

Para tener en cuenta (Calvo):

- Usar variables de entorno propias para la carpeta 03 en Postman (`orderNumberLoad`, `activationPasswordLoad`, etc.) para no interferir con las variables de las carpetas 01 y 02.
- Las aserciones sobre `reason` deben buscar fragmentos con `include` (ej. `"menor que la anterior"`, `"debe ser mayor que 0"`).
- Recordar que tanto para guardadas como descartadas el código HTTP es 200 (la diferencia está en el campo booleano `saved`).

Estado de la Parte 3 (27/10/2026)

Paso	Commit / archivos	Estado
A — Postman, carpeta 03	`f07627a` `test(postman): agrega la carpeta de la Fase 3 (sistema de carga)`	Hecho
C — Arreglo de H-14 (autorizado por Calvo)	`4ed54c8` `fix(business): usa saveAndFlush en la activación para capturar errores de base dentro del try`	Hecho
G — Decisión 11 (aprobada por Calvo)	`9cc4a05` (repositorio), `935d11e` (negocio), `191e327` (controller), `041308a` (Postman)	Hecho
B y C — Pruebas y auditoría	Resultados en `docs/fase-3-carga-tiempo-real.md` (secciones 7 a 10)	Hecho
D — Informe y guía	`docs/fase-3-carga-tiempo-real.md`	Hecho

Paso	Commit / archivos	Estado
E — Correcciones de documentación	`CLAUDE.md`, `docs/plan-de-trabajo.md`, `docs/guia-estudio-src.md`, `docs/fase-2-pesaje-inicial.md`, este bloque, `docs/pdf/`	Hecho
F — Pull Request	PR #7	Hecho: mergeado en `main`

Qué quedó hecho:

- **Carpeta Postman 03** con 44 pedidos y 114 aserciones, variables propias (`orderNumberLoad`, `orderNumberLoadPending`, `orderNumberLoadInactive`, `activationPasswordLoad`, `wrongPasswordLoad`, `activationDateTimeLoad`) y aserciones de mensajes con `include`. Los `git diff` de la colección solo agregan líneas: las carpetas 00 a 02 no cambiaron.
- **H-14 corregido:** `OrderBusiness.activate` usa `saveAndFlush`. Es un archivo de la Parte 1: el cambio lo autorizó Calvo y hay que avisarle a Gonzalez.
- **Decisión 11:** `GET /api/v1/orders/{orderNumber}/details` paginado (`size` 20, máximo 100, orden fijo por `id`, `PagedModel`, 404). La división la preveía con aprobación de Martín; la aprobó Calvo.
- **Informe técnico, matriz de auditoría y guía A–J** (16 preguntas) en `docs/fase-3-carga-tiempo-real.md`.
- **Correcciones del apartado E**; las filas duplicadas de la sección 15 de la guía de `src` se quitaron. PDF regenerados.

Verificado (27/10/2026, MySQL real `iw3-gas-mysql`):

Verificación	Resultado
Arranque sobre una base con el esquema de la Fase 1 y una orden	Hibernate agregó 10 columnas nulables y creó `order_details`; la orden previa quedó intacta
`npx newman@6 run docs/postman/IW3-Gas-Terminal.postman_collection.json`	65 pedidos, **160 aserciones, 0 fallas** (las 46 de las carpetas 00 a 02 incluidas)
`./mvnw -B test`	11 tests, 0 fallas, `BUILD SUCCESS`
`SELECT COUNT(*) FROM order_details WHERE id_order = <id>`	4 (las lecturas guardadas); 0 en las órdenes en estado 1 y sin activar
Orden de comprobaciones (D7) con dos errores a la vez	Según lo decidido (script de auditoría: 18 PASS, 0 FAIL)
Estado 3 simulado en la base local	Lecturas y activación → 409, sin escribir nada
Atomicidad (`CHECK` temporal)	500 con `StandartResponse`; detalle y cabecera revertidos
Falla al confirmar la activación (*trigger* temporal)	Antes: 500 con el formato de Spring (**H-14**). Después de `4ed54c8`: 500 con `StandartResponse`
100 lecturas seguidas	79,6 ms promedio, sin degradación
Contraseña en logs y respuestas	No aparece

No verificado: la consigna original en PDF (no está disponible). Swagger UI en el navegador (solo `/v3/api-docs`).

Para tener en cuenta:

- **Avisar a Martín González:** se tocó su `OrderBusiness.activate` (H-14, `4ed54c8`) y se implementó la decisión 11, que la división preveía que aprobara él.
- **Pendientes del dueño del repo (`joacoContreras`):** el ruleset de `main` pide 1 aprobación en vez de 3 y no borra las ramas al mergear.
- **Datos de prueba:** las pruebas de la base local se revirtieron (estado, `CHECK` y *trigger*). Las órdenes de prueba de newman y de la auditoría se borraron al final.